These are the 10 test cases I done from the 100 case studies

CASE STUDY 1(number:12)

1. aim: Attendance Recovery Computation.py

2. objective:

Calculate current attendance percentages and determine the minimum consecutive classes required to reach an eligibility threshold under different future attendance scenarios.

3. python code:

```
attended=int(input("enter classes attended: "))
conducted=int(input("enter classes conducted: "))
print("required 75% attendance")
print(f'your attendance is {100*attended/conducted:.2f} ')
to_attend=0
present=100*attended//conducted
if present<75:
    while present<75:
        to_attend=to_attend+1
        present=((to_attend+attended)/(conducted+to_attend))*100
    print(f"you need to attend {to_attend} upcoming classes to aquire required
attendance")
else:
    print(f'you have required attendance')
```

4. screen shots:

```
enter classes attended: 14
enter classes conducted: 20
required 75% attendance
your attendance is 70.00
you need to attend 4 upcoming classes to aquire required attendance
```

5.algorithm:

Step 1: start

Step 2: read classes attended=attended

Step 3: read classes conducted=conducted

Step 4: display “required 75% attendance”

Step 5: display 100*attended/conducted:.2f

Step 6: take to_attend=0

Step 7: take present=100*attended//conducted

Step 8: if present<75: then

Step 9: while present<75:

Step 10:take to_attend=to_attend+1

Step 11: take present=((to_attend+attended)/(conducted+to_attend))*100

Step 12: end while loop

Step 13: display to_attend

Step14: else display you have required percentage

Step 15: stop

6. pseudocode:

START

READ attended

READ conducted

DISPLAY "required 75% attendance"

DISPLAY (100 * attended / conducted)

SET to_attend = 0

SET present = 100 * attended // conducted

IF present < 75 THEN

WHILE present < 75 DO

SET to_attend = to_attend + 1

SET present = ((to_attend + attended) / (conducted + to_attend)) * 100

END WHILE

DISPLAY "you need to attend " + to_attend + " upcoming classes to acquire required attendance"

ELSE

DISPLAY "you have required attendance"

END IF

END

CASE STUDY 2(number:3)

1. aim: Hospital Billing and Patient Classification.py

2. objective:

Process consultation, diagnostic, and treatment charges, apply conditional concessions, validate billing constraints, and classify patients according to a supplied payment policy.

3. python code:

```
patients = []
n = int(input("Enter number of patients: "))
for i in range(n):
    print("Enter details for Patient",i+1)
    name = input("Enter patient name: ")
    consultation = float(input("Enter consultation charges: "))
    diagnostic = float(input("Enter diagnostic charges: "))
    treatment = float(input("Enter treatment charges: "))
    total = consultation + diagnostic + treatment
    if total>=10000:
        discount=(total/100)*20
    elif total>=5000:
        discount=(total/100)*10
    else:
        discount = 0
    total_bill = total - discount
    if total_bill >= 10000:
        category = "High Billing"
    elif total_bill >= 5000:
        category = "Medium Billing"
    else:
        category = "Low Billing"
    patients.append([name, total, discount, total_bill, category])
for i in patients:
    print("Patient Name:", i[0])
    print("Total Charges: Rs.", i[1])
    print("Concession: Rs.", i[2])
    print("Final Bill: Rs.", i[3])
    print("Classification:", i[4])
print("Total patients processed:", len(patients))
```

4. screen shots:

```
Enter number of patients: 2
Enter details for Patient 1
Enter patient name: mustafavali
Enter consultation charges: 1000
Enter diagnostic charges: 2000
Enter treatment charges: 3000
Enter details for Patient 2
Enter patient name: karthik
Enter consultation charges: 2000
Enter diagnostic charges: 1000
Enter treatment charges: 3500
Patient Name: mustafavali
Total Charges: Rs. 6000.0
Concession: Rs. 600.0
Final Bill: Rs. 5400.0
Classification: Medium Billing
Patient Name: karthik
Total Charges: Rs. 6500.0
Concession: Rs. 650.0
Final Bill: Rs. 5850.0
Classification: Medium Billing
Total patients processed: 2
```

5.algorithm:

Step 1: start

Step 2: take patients=[]

Step 3: read no of patients=n

Step 4: for I in range(n):

Step 5: display "Enter details for Patient",i+1

Step 6: read name of patient=name

Step 7: read consultation cost=consultation

Step 8: read diagnostic cost=diagnostic

Step 9: read treatment cost=treatment

Step10: take total=consulation+diagnostic+treatment

Step 11: if total>=10000: then

Step 12: discount=(total/100)*20

Step 13: elif total>=5000: then

Step 14: discount=(total/100)*10

Step 15: else discount=0

Step 16: take total_bill=total-discount

Step 17: if total_bill>=10000: then

Step 18: take category = "High Billing"

Step 19: e;if total_bill>=5000: then

Step 20: take category = "Medium Billing"

Step 21: else take category = "Low Billing"

Step 22: append([name,total,discount,total_bill,category]) into patients

Step 23: end for loop

Step 24: for j in patients

Step 25: display name, total, discount, total_bill, category

Step 26: end for lopp and diplay len(patients)

Step 27: stop

6. pseudocode:

START

SET patients = []

READ n

FOR i FROM 0 TO n - 1 DO

DISPLAY "Enter details for Patient", i + 1

READ name

READ consultation

READ diagnostic

READ treatment

SET total = consultation + diagnostic + treatment

IF total >= 10000 THEN

SET discount = (total / 100) * 20

```
ELSE IF total >= 5000 THEN
SET discount = (total / 100) * 10
ELSE
SET discount = 0
END IF
SET total_bill = total - discount
IF total_bill >= 10000 THEN
SET category = "High Billing"
ELSE IF total_bill >= 5000 THEN
SET category = "Medium Billing"
ELSE
SET category = "Low Billing"
END IF
APPEND [name, total, discount, total_bill, category] TO patients
END FOR
FOR EACH patient IN patients DO
DISPLAY "Patient Name:", patient[0]
DISPLAY "Total Charges: Rs.", patient[1]
DISPLAY "Concession: Rs.", patient[2]
DISPLAY "Final Bill: Rs.", patient[3]
DISPLAY "Classification:", patient[4]
END FOR
DISPLAY "Total patients processed:", LENGTH(patients)
END
```

CASE STUDY 3(number:58)

1. aim: Library Borrowing and Fine Reconciliation.py

2. objective:

Analyse tuples representing book identifiers, borrowing periods, and return delays; calculate fines, identify overdue records, and reconcile book availability.

3. python code:

```
valid_books=["book1","book2","book3","book4","book5"]
books=['book1'*3,'book2'*3,'book3'*3,'book4'*3,'book5'*3]
print("available actions: take , return")
request=input("what is your request: ")
if request=="take":
    name=input("what book do you want: ")
    if name in books:
        books.remove(name)
        print("return within 5 days!")
    else:
        print("this is not available")
elif request=="return":
    name=input("what book are you returning: ")
    if name in valid_books:
        books.append(name)
    time=int(input("for how many days did you take this book?: "))
    if time>5:
        fine=50
        for i in range(5,time):
            fine=fine+50
    else:
        fine=0
    print(f"you need to pay {fine}")
```

4. screen shots:

```
available actions: take , return
what is your request: return
what book are you returning: book1
for how many days did you take this book?: 7
you need to pay 150
```

5.algorithm:

Step 1: start

Step 2: take valid_books= ["book1","book2","book3","book4","book5"]

Step 3: take books=['book1'*3,'book2'*3,'book3'*3,'book4'*3,'book5'*3]

Step 4: display "available actions: take , return"

Step 5: read the user request= request

Step 5: if request==”take”: then

Step 6: read name of the book=name

Step 7: if name in books: then

Step 8: books.append(name)

Step 9: display "return within 5 days!"

Step 10: else: display "this is not available"

Step 11: stop inner if and outer if and elif request==”return”

Step 12: take name of book=name

Step 13: if name in valid_books: then

Step 14: books.append(name)

Step 15: stop inner if and take how many days=time

Step 16: if time>5:

Step 17: fine=50

Step 18: for I in range(5,time):

Step 19: fine=fine+50

Step 20: stop for loop

Step 21: else: fine=0

Step 22: display fine

Step 23: stop

6. pseudocode:

START

SET valid_books = ["book1", "book2", "book3", "book4", "book5"]

SET books = ["book1"*3, "book2"*3, "book3"*3, "book4"*3, "book5"*3]

DISPLAY "available actions: take , return"

READ request

IF request == "take" THEN

READ name

IF name IN books THEN

```
REMOVE name FROM books
DISPLAY "return within 5 days!"
ELSE
DISPLAY "this is not available"
END IF
ELSE IF request == "return" THEN
READ name
IF name IN valid_books THEN
APPEND name TO books
END IF
READ time
IF time > 5 THEN
SET fine = 50
FOR i FROM 5 TO time - 1 DO
SET fine = fine + 50
END FOR
ELSE
SET fine = 0
END IF
DISPLAY "you need to pay " + fine
END IF
END
```

CASE STUDY 4(number:58)

1. aim: Multi-Product Shopping Budget Constraint.py

2. objective:

Evaluate a shopping basket containing multiple products and quantities, apply discount conditions, and determine which purchases must be reduced to remain within a fixed budget.

3. python code:

```
products=[]
total=0
budget=float(input("Enter your shopping budget: Rs."))
n=int(input("Enter number of products: "))
for i in range(n):
    name=input("Enter product name: ")
    price=float(input("Enter price: ₹"))
    quantity=int(input("Enter quantity: "))
    cost=price*quantity
    total=total+cost
    products.append([name, price, quantity, cost])
for j in products:
    print(j[0], "- Rs.", j[1], "x", j[2], "= Rs.", j[3])
print("Total bill: Rs.", total)
if total <= budget:
    print("Shopping is within your budget.")
else:
    print("Shopping exceeds your budget.")
    print("Amount exceeded: ₹", total - budget)
```

4. screen shots:

```
Enter your shopping budget: Rs.10000
Enter number of products: 5
Enter product name: shirt
Enter price: ₹500
Enter quantity: 4
Enter product name: pants
Enter price: ₹700
Enter quantity: 3
Enter product name: bags
Enter price: ₹1400
Enter quantity: 2
Enter product name: shoe
Enter price: ₹600
Enter quantity: 1
Enter product name: watch
Enter price: ₹1000
Enter quantity: 1
shirt - Rs. 500.0 x 4 = Rs. 2000.0
pants - Rs. 700.0 x 3 = Rs. 2100.0
bags - Rs. 1400.0 x 2 = Rs. 2800.0
shoe - Rs. 600.0 x 1 = Rs. 600.0
watch - Rs. 1000.0 x 1 = Rs. 1000.0
Total bill: Rs. 8500.0
Shopping is within your budget.
```

5.algorithm:

Step 1: start

Step 2: take products=[]

Step 3: take total=0

Step 4: read shopping budget=budget

Step 5: read number of products=n

Step 6: for i in range(n):

Step 7: read product name=name

Step 8: read price=price

Step 9: read quantity=quantity

Step 10: take cost=price*quantity

Step 11: take total=total+cost

Step 12: append [name, price, quantity, cost] into products

Step 13: end for loop

Step 14: for j in products:

Step 15: display j[0], "- Rs.", j[1], "x", j[2], "= Rs.", j[3]

Step 16: end for loop

Step 17: display "Total bill: Rs.", total

Step 18: if total <= budget: then

Step 19: display "Shopping is within your budget."

Step 20: else:

Step 21: display "Shopping exceeds your budget."

Step 22: display "Amount exceeded: ₹", total – budget

Step 23: stop

6. pseudocode:

START

SET products = []

SET total = 0

READ budget

READ n

FOR i FROM 0 TO n - 1 DO

READ name

READ price

READ quantity

SET cost = price * quantity

SET total = total + cost

APPEND [name, price, quantity, cost] TO products

END FOR

FOR EACH item IN products DO

DISPLAY item[0], "- Rs.", item[1], "x", item[2], "= Rs.", item[3]

END FOR

DISPLAY "Total bill: Rs.", total

IF total <= budget THEN

DISPLAY "Shopping is within your budget."

ELSE

DISPLAY "Shopping exceeds your budget."

DISPLAY "Amount exceeded: ₹", total - budget

END IF

END

CASE STUDY 5(number:70)

1. aim: Multi-Stage Scholarship Shortlisting.py

2. objective:

Apply minimum academic eligibility, attendance requirements, income conditions, and ranking rules to select candidates within a fixed scholarship quota.

3. python code:

```
details=[]
details.append(input("Enter your name: "))
details.append(int(input("Enter your marks percentage: ")))
details.append(int(input("Enter your attendance percentage: ")))
details.append(int(input("Enter your income: ")))
if details[1]>60 and details[2]>70:
        if details[3]>10000:
            category='Merit-cum-Means scholarship'
        else:
            category='Merit-Based scholarship'
else:
    if details[3]>10000:
        category='Need-Based scholarship'
    else:
        category='no category'
print(f'the student can apply for scholarship under {category}')
```

4. screen shots:

```
Enter your name: mustafavali
Enter your marks percentage: 80
Enter your attendance percentage: 70
Enter your income: 70000
the student can apply for scholarship under Need-Based scholarship
```

5.algorithm:

Step 1: start

Step 2: take details=[]

Step 3: read name and append to details

Step 4: read marks percentage and append to details

Step 5: read attendance percentage and append to details

Step 6: read income and append to details

Step 7: if details[1]>60 and details[2]>70: then

Step 8: if details[3]>10000: then

Step 9: take category='Merit-cum-Means scholarship'

Step 10: else:

Step 11: take category='Merit-Based scholarship'

Step 12: else:

Step 13: if details[3]>10000: then

Step 14: take category='Need-Based scholarship'

Step 15: else: Step 16: take category='no category'

Step 17: stop inner if-else and outer if-else

Step 18: display f'the student can apply for scholarship under {category}'

Step 19: stop

6. pseudocode:

START

SET details = []

READ name AND APPEND TO details

READ marks AND APPEND TO details

READ attendance AND APPEND TO details

READ income AND APPEND TO details

IF details[1] > 60 AND details[2] > 70 THEN

IF details[3] > 10000 THEN

SET category = 'Merit-cum-Means scholarship'

ELSE

SET category = 'Merit-Based scholarship'

END IF

ELSE

IF details[3] > 10000 THEN

SET category = 'Need-Based scholarship'

ELSE

SET category = 'no category'

END IF

END IF

DISPLAY "the student can apply for scholarship under " + category

END

CASE STUDY 6(number:29)

1. aim: Password Policy Validation Engine.py

2. objective:

Evaluate sample password strings against minimum length, character-category, repetition, and prohibited-substring rules; explain each failed condition.

3. python code:

```
password1=0
while password1!="strong":
    up=0
    low=0
    spe=0
    num=0
    length=0
    password=input("enter password: ")
    for i in password:
        if 48<=ord(i)<=57:
            num+=1
        elif 65<=ord(i)<=90:
            up+=1
        elif 97<=ord(i)<=122:
            low+=1
        else:
            spe+=1
        length+=1
    if length>8:
        if length<20:
            if num>0:
                if low>0:
                    if up>0:
                        if spe>0:
                            print("your password is strong")
                            password1="strong"
                        else:
                            print("password doesn't have special characters")
                            password1="not strong"
                    else:
                        print("password doesn't have upper characters")
                        password1="not strong"
                else:
                    print("password doesn't have lower characters")
                    password1="not strong"
            else:
                print("password doesn't have numbers")
                password1="not strong"
        else:
            print("password id long")
```

```
            password1="not strong"
    else:
        print("password is short")
        password1="not strong"
```

4. screen shots:

```
enter password: Asdf2322
password is short
enter password: A@#$adjdns
password doesn't have numbers
enter password: Ajefsj234@!
your password is strong
```

5.algorithm:

Step 1: start

Step 2: take password1=0

Step 3: while password1!="strong":

Step 4: take up=0, low=0, spe=0, num=0, length=0

Step 5: read password=password

Step 6: for i in password:

Step 7: if 48<=ord(i)<=57: then num+=1

Step 8: elif 65<=ord(i)<=90: then up+=1

Step 9: elif 97<=ord(i)<=122: then low+=1

Step 10: else: spe+=1

Step 11: take length+=1

Step 12: end for loop

Step 13: if length>8: then

Step 14: if length<20: then

Step 15: if num>0: then

Step 16: if low>0: then

Step 17: if up>0: then

Step 18: if spe>0: then

Step 19: display "your password is strong"

Step 20: take password1="strong"

Step 21: else: display "password doesn't have special characters", take password1="not strong"

Step 22: else: display "password doesn't have upper characters", take password1="not strong"

Step 23: else: display "password doesn't have lower characters", take password1="not strong"

Step 24: else: display "password doesn't have numbers", take password1="not strong" Step 25: else: display "password is long", take password1="not strong"

Step 26: else: display "password is short", take password1="not strong"

Step 27: end while loop

Step 28: stop

6. pseudocode:

START

SET password1 = 0

WHILE password1 != "strong" DO

SET up = 0, low = 0, spe = 0, num = 0, length = 0

READ password

FOR EACH char IN password DO

IF ASCII(char) >= 48 AND ASCII(char) <= 57 THEN

SET num = num + 1

ELSE IF ASCII(char) >= 65 AND ASCII(char) <= 90 THEN

SET up = up + 1

ELSE IF ASCII(char) >= 97 AND ASCII(char) <= 122 THEN

SET low = low + 1

ELSE

SET spe = spe + 1

END IF

SET length = length + 1

END FOR

IF length > 8 THEN

```
IF length < 20 THEN
IF num > 0 THEN
IF low > 0 THEN
IF up > 0 THEN
IF spe > 0 THEN
DISPLAY "your password is strong"
SET password1 = "strong"
ELSE
DISPLAY "password doesn't have special characters"
SET password1 = "not strong"
END IF
ELSE
DISPLAY "password doesn't have upper characters"
SET password1 = "not strong"
END IF
ELSE
DISPLAY "password doesn't have lower characters"
SET password1 = "not strong"
END IF
ELSE
DISPLAY "password doesn't have numbers"
SET password1 = "not strong"
END IF
ELSE
DISPLAY "password id long"
SET password1 = "not strong"
END IF
ELSE
```

DISPLAY "password is short"

SET password1 = "not strong"

END IF

END WHILE

END

CASE STUDY 7(number:2)

1. aim: Progressive Electricity Billing.py

2. objective:

Calculate household bills using multiple consumption slabs, fixed charges, surcharges, and rebates, then compare bills across several months and identify unusual increases.

3. python code:

```
n = int(input("Enter number of households: "))
bills = []
for i in range(n):
    print("Household", i + 1)
    name = input("Enter household name: ")
    units = float(input("Enter electricity units consumed: "))
    if units <= 100:
        bill=units*2
    elif units<=200:
        bill= units*3
    elif units<=300:
        bill=units*5
    else:
        bill=units*7
    fixed_charge=50
    total_bill=bill+fixed_charge
    bills.append([name, units, total_bill])
for i in bills:
    print("Household:", i[0])
    print("Units consumed:", i[1])
    print("Energy charge: Rs.", i[2] - 50)
    print("Fixed charge: Rs. 50")
    print("Total bill: Rs.", i[2])
highest=bills[0]
for j in bills:
    if j[2]>highest[2]:
        highest=j
print("Highest bill:", highest[0])
print("Amount: Rs.", highest[2])
```

4. screen shots:

```
Enter number of households: 2
Household 1
Enter household name: mustafavali
Enter electricity units consumed: 223
Household 2
Enter household name: srinivas
Enter electricity units consumed: 305
Household: mustafavali
Units consumed: 223.0
Energy charge: Rs. 1115.0
Fixed charge: Rs. 50
Total bill: Rs. 1165.0
Household: srinivas
Units consumed: 305.0
Energy charge: Rs. 2135.0
Fixed charge: Rs. 50
Total bill: Rs. 2185.0
Highest bill: srinivas
Amount: Rs. 2185.0
```

5.algorithm:

Step 1: start

Step 1: start

Step 2: read number of households=n

Step 3: take bills=[]

Step 4: for i in range(n):

Step 5: display "Household", i + 1

Step 6: read household name=name

Step 7: read electricity units consumed=units

Step 8: if units <= 100: then bill=units*2

Step 9: elif units <= 200: then bill=units*3

Step 10: elif units <= 300: then bill=units*5

Step 11: else: bill=units*7

Step 12: take fixed_charge=50

Step 13: take total_bill=bill+fixed_charge

Step 14: append [name, units, total_bill] into bills

Step 15: end for loop

Step 16: for i in bills:

Step 17: display "Household:", i[0]

Step 18: display "Units consumed:", i[1]

Step 19: display "Energy charge: Rs.", i[2] - 50

Step 20: display "Fixed charge: Rs. 50"

Step 21: display "Total bill: Rs.", i[2]

Step 22: end for loop

Step 23: take highest=bills[0]

Step 24: for j in bills:

Step 25: if j[2]>highest[2]: then highest=j

Step 26: end for loop

Step 27: display "Highest bill:", highest[0]

Step 28: display "Amount: Rs.", highest[2]

Step 29: stop

6. pseudocode:

START

READ n

SET bills = []

FOR i FROM 0 TO n - 1 DO

DISPLAY "Household", i + 1

READ name

READ units

IF units <= 100 THEN

SET bill = units * 2

ELSE IF units <= 200 THEN

```
SET bill = units * 3
ELSE IF units <= 300 THEN
SET bill = units * 5
ELSE
SET bill = units * 7
END IF
SET fixed_charge = 50
SET total_bill = bill + fixed_charge
APPEND [name, units, total_bill] TO bills
END FOR
FOR EACH h IN bills DO
DISPLAY "Household:", h[0]
DISPLAY "Units consumed:", h[1]
DISPLAY "Energy charge: Rs.", h[2] - 50
DISPLAY "Fixed charge: Rs. 50"
DISPLAY "Total bill: Rs.", h[2]
END FOR
SET highest = bills[0]
FOR EACH h IN bills DO
IF h[2] > highest[2] THEN
SET highest = h
END IF
END FOR
DISPLAY "Highest bill:", highest[0]
DISPLAY "Amount: Rs.", highest[2]
END
```

CASE STUDY 8(number:57)

1. aim: Smart Canteen Order and Stock Management.py

2. objective:

Process successive customer orders, calculate bills, update available inventory, reject unavailable quantities, and summarize daily sales and remaining stock.

3. python code:

```
rice=20
eggs=30
noodles=10
chicken=2000
items=['fried rice','noodles','chicken rice']
special=['eggs','chicken']
print("available items are: ")
for i in items:
    print(i)
print("additional:")
for i in special:
    print(i)
nxt_order=1
while nxt_order!="no":
    if rice<1 or eggs<1 or noodles<1 or chicken<200:
        nxt_order=0
        print("out of stock")
        break
    order=input("what you want to order: ").lower()
    extra=input("specialization?(0 if no need): ").lower()
    if order=="fried rice":
        rice=rice-1
        if extra=="egg":
            eggs=eggs-1
        elif extra=="chicken":
            chicken=chicken-200
    elif order=="chicken rice":
        rice=rice-1
        if extra=="egg":
            eggs=eggs-1
    elif order=="noodles":
        noodles=noodles-1
        if extra=="egg":
            eggs=eggs-1
        elif extra=="chicken":
            chicken=chicken-200
    else:
        print("there no such type of item")
    nxt_order=input("do you want next order('no' if dont need): ")
```

4. screen shots:

```
available items are:
fried rice
noodles
chicken rice
additional:
eggs
chicken
what you want to order: fried rice
specialization?(0 if no need): chicken
do you want next order('no' if dont need): yes
what you want to order: noodles
specialization?(0 if no need): egg
do you want next order('no' if dont need): no
```

5.algorithm:

Step 1: start

Step 1: start

Step 2: take rice=20, eggs=30, noodles=10, chicken=2000

Step 3: take items=['fried rice','noodles','chicken rice']

Step 4: take special=['eggs','chicken']

Step 5: display "available items are:" and print items

Step 6: display "additional:" and print special

Step 7: take nxt_order=1

Step 8: while nxt_order!="no":

Step 9: if rice<1 or eggs<1 or noodles<1 or chicken<200: then

Step 10: take nxt_order=0

Step 11: display "out of stock"

Step 12: break while loop

Step 13: read order=order (in lowercase)

Step 14: read extra=extra (in lowercase)

Step 15: if order=="fried rice": then

Step 16: take rice=rice-1

Step 17: if extra=="egg": then eggs=eggs-1

Step 18: elif extra=="chicken": then chicken=chicken-200

Step 19: elif order=="chicken rice": then

Step 20: take rice=rice-1

Step 21: if extra=="egg": then eggs=eggs-1

Step 22: elif order=="noodles": then

Step 23: take noodles=noodles-1

Step 24: if extra=="egg": then eggs=eggs-1

Step 25: elif extra=="chicken": then chicken=chicken-200

Step 26: else: display "there no such type of item"

Step 27: read nxt_order

Step 28: end while loop

Step 29: stop

6. pseudocode:

START

SET rice = 20, eggs = 30, noodles = 10, chicken = 2000

SET items = ['fried rice', 'noodles', 'chicken rice']

SET special = ['eggs', 'chicken']

DISPLAY "available items are: "

FOR EACH item IN items DO

DISPLAY item

END FOR

DISPLAY "additional:"

FOR EACH s IN special DO

DISPLAY s

END FOR

SET nxt_order = 1

WHILE nxt_order != "no" DO

```
IF rice < 1 OR eggs < 1 OR noodles < 1 OR chicken < 200 THEN
SET nxt_order = 0
DISPLAY "out of stock"
BREAK
END IF
READ order TO LOWERCASE
READ extra TO LOWERCASE
IF order == "fried rice" THEN
SET rice = rice - 1
IF extra == "egg" THEN
SET eggs = eggs - 1
ELSE IF extra == "chicken" THEN
SET chicken = chicken - 200
END IF
ELSE IF order == "chicken rice" THEN
SET rice = rice - 1
IF extra == "egg" THEN
SET eggs = eggs - 1
END IF
ELSE IF order == "noodles" THEN
SET noodles = noodles - 1
IF extra == "egg" THEN
SET eggs = eggs - 1
ELSE IF extra == "chicken" THEN
SET chicken = chicken - 200
END IF
ELSE
DISPLAY "there no such type of item"
```

END IF

READ nxt_order

END WHILE

END

CASE STUDY 9(number:15)

1. aim: Smart Parking Fee Management.py

2. objective:

Calculate charges from entry and exit times, vehicle categories, overnight parking, daily limits, and penalty conditions while validating time inputs.

3. python code:

```
A=["car",'bike','truck','bicycle']
print(f'available slots for vehicles {A}')
parking_vehicle=input("enter your parking vehicle: ").lower().strip()
time=int(input("enter how many hours(0 if less than 1 hour): "))
if parking_vehicle =='car':
    maximum=10
    fee=100
elif parking_vehicle =='bike':
    maximum=5
    fee=50
elif parking_vehicle =='truck':
    maximum=15
    fee=200
else:
    maximum=0
    fee=0
if time>maximum:
    print(f'you need to pay extra 1000')
else:
    for i in range(time):
        fee=fee+20*(fee/100)
print(f'your total fee is {int(fee)}')
```

4. screen shots:

```
available slots for vehicles ['car', 'bike', 'truck', 'bicycle']
enter your parking vehicle: car
enter how many hours(0 if less than 1 hour): 5
your total fee is 248
```

5.algorithm:

Step 1: start

Step 1: start

Step 2: take A=["car", 'bike', 'truck', 'bicycle']

Step 3: display "available slots for vehicles", A

Step 4: read parking vehicle=parking_vehicle (lowercase and stripped)

Step 5: read time=time

Step 6: if parking_vehicle=='car': then maximum=10, fee=100

Step 7: elif parking_vehicle=='bike': then maximum=5, fee=50

Step 8: elif parking_vehicle=='truck': then maximum=15, fee=200

Step 9: else: maximum=0, fee=0

Step 10: if time>maximum: then

Step 11: display "you need to pay extra 1000"

Step 12: else:

Step 13: for i in range(time):

Step 14: fee=fee+20*(fee/100)

Step 15: end for loop

Step 16: display "your total fee is", int(fee)

Step 17: stop

6. pseudocode:

START

SET A = ["car", "bike", "truck", "bicycle"]

DISPLAY "available slots for vehicles", A

READ parking_vehicle TO LOWERCASE AND STRIPPED

```
READ time
IF parking_vehicle == 'car' THEN
SET maximum = 10
SET fee = 100
ELSE IF parking_vehicle == 'bike' THEN
SET maximum = 5
SET fee = 50
ELSE IF parking_vehicle == 'truck' THEN
SET maximum = 15
SET fee = 200
ELSE
SET maximum = 0
SET fee = 0
END IF
IF time > maximum THEN
DISPLAY "you need to pay extra 1000"
ELSE
FOR i FROM 0 TO time - 1 DO
SET fee = fee + 20 * (fee / 100)
END FOR
END IF
DISPLAY "your total fee is", INTEGER(fee)
END
```

CASE STUDY 10(number:1)

1. aim: University Scholarship Allocation.py

2. objective:

Design a system that evaluates applicants using aggregate marks, subject-wise minimum scores, attendance, and income criteria, ranks eligible students, and resolves ties using predefined rules.

3. python code:

```
students=[]
marks=[]
attendance=[]
income=[]
for i in range(int(input("Enter how many students: "))):
    students.append((input("Enter your name: ")))
    marks.append(int(input("Enter your marks percentage: ")))
    attendance.append(int(input("Enter your attendance percentage: ")))
    income.append(int(input("Enter your income: ")))
eligible=[]
for j in marks:
    if j>60:
        a=marks.index(j)
        eligible.append(students[a])
print("The eligible students are: ")
for k in eligible:
    print(k,end=" ")
    b=students.index(k)
    if income[b]<80000:
        print("with scholarship of 70%")
    elif income[b]>120000:
        print("with scholarship of 50%")
    else:
        print("with scholarship of 30%")
```

4. screen shots:

```
Enter how many students: 2
Enter your name: mustafavali
Enter your marks percentage: 70
Enter your attendance percentage: 75
Enter your income: 70000
Enter your name: karthik
Enter your marks percentage: 80
Enter your attendance percentage: 60
Enter your income: 80000
The eligible students are:
mustafavali with scholarship of 70%
karthik with scholarship of 30%
```

5.algorithm:

Step 1: start

Step 1: start

Step 2: take students=[], marks=[], attendance=[], income=[]

Step 3: read number of students and loop that many times:

Step 4: read student name and append to students

Step 5: read marks percentage and append to marks

Step 6: read attendance percentage and append to attendance

Step 7: read income and append to income

Step 8: end for loop

Step 9: take eligible=[]

Step 10: for j in marks:

Step 11: if j>60: then

Step 12: take a=marks.index(j)

Step 13: append students[a] into eligible

Step 14: end for loop

Step 15: display "The eligible students are:"

Step 16: for k in eligible:

Step 17: display k (with end=" ")

Step 18: take b=students.index(k)

Step 19: if income[b]<80000: then display "with scholarship of 70%"

Step 20: elif income[b]>120000: then display "with scholarship of 50%"

Step 21: else: display "with scholarship of 30%"

Step 22: end for loop

Step 23: stop

6. pseudocode:

START

SET students = []

SET marks = []

SET attendance = []

SET income = []

READ n

FOR i FROM 0 TO n - 1 DO

```
READ name AND APPEND TO students
READ mark_val AND APPEND TO marks
READ att_val AND APPEND TO attendance
READ inc_val AND APPEND TO income
END FOR
SET eligible = []
FOR EACH score IN marks DO
IF score > 60 THEN
SET idx = INDEX_OF(score, marks)
APPEND students[idx] TO eligible
END IF
END FOR
DISPLAY "The eligible students are: "
FOR EACH student IN eligible DO
DISPLAY student WITHOUT NEWLINE
SET idx = INDEX_OF(student, students)
IF income[idx] < 80000 THEN
DISPLAY "with scholarship of 70%"
ELSE IF income[idx] > 120000 THEN
DISPLAY "with scholarship of 50%"
ELSE
DISPLAY "with scholarship of 30%"
END IF
END FOR
END
```